

```
(.venv) PS C:\Users\zrun2\OneDrive\Education\2025-2026 12TH Grade\4th COP 2373 - Melichar> ipython
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
Type 'copyright', 'credits' or 'license' for more information
IPython 9.9.0 -- An enhanced Interactive Python. Type '?' for help.
Tip: IPython 9.0+ has hooks to integrate AI/LLM completions.

In [1]: print(10, 20, 30, end='.', sep=',')
10,20,30.
In [2]: def pr_named_vals(**kwargs):
...:     for k in kwargs:
...:         print(k, ': ', kwargs[k])
...:

In [3]: pr_named_vals(a=10, b=20, c=30)
a : 10
b : 20
c : 30

In [4]: def pr_vals_2(*args, **kwargs):
...:     for i in args:
...:         print(i)
...:     for k in kwargs:
...:         print(k, ': ', kwargs[k])
...:

In [5]: pr_vals_2(1, 2, 3, -4, a=100, b=200)
1
2
3
-4
a : 100
b : 200

In [6]:
```

Advantages:

1. ****kwargs** allows a function to accept any number of keyword arguments, making functions more flexible if you don't know exactly what arguments will be passed in advance.
2. ****kwargs** can be passed to other functions, making wrapper functions more versatile and easier to create.
3. When a function has many optional parameters with default values, ****kwargs** allows you to pass only the ones you need to override, making function calls cleaner.

Disadvantages:

1. Using ****kwargs** in functions makes them less readable since its not clear what the expected argument by the function are, requiring that you read the function itself to understand.

2. Using `**kwargs` makes debugging code harder since any misspelled argument names may not raise an error immediately if the function just iterates over the kwargs dictionary, making the code harder to debug.
3. Using `**kwargs` can make autocomplete in IDEs difficult, as there aren't pre-existing structures of which arguments a function should take, increasing the time it takes to code a program.